# ReleaseShield AI

Architecture, data pipeline and Sprint 1 plan | Team Algorithm

## 1. Understanding the project

### 1.1 The problem

Small engineering teams run many security scanners side by side: Trivy (containers and dependencies), Semgrep (code patterns, called SAST), OWASP ZAP (running website, called DAST) and Gitleaks (hardcoded secrets). Each tool writes its own report in its own format. The result is **scanner noise and triage fatigue**: the same bug appears in several reports, nobody knows who owns it, nobody knows if it was already seen, and developers stop looking.

### 1.2 What we are building

A lightweight control plane for CI/CD that does five jobs for the developer:

- **Run** the open-source scanners (we do not write our own scanners).
- **Normalize** every result into one standard format called SARIF.
- **Deduplicate and correlate**: remove repeats, and link findings that describe the same bug.
- **Assign an owner** for each finding using the repository's CODEOWNERS file.
- **Suggest a fix with AI in plain language**, then check the fix by running the scanner again.

The user is a developer on a small team, not a compliance officer. The product must stay simple, quick and developer-friendly.

### 1.3 What the reviewer said (feedback summary)

**Strengths:** good Scrum framing (Sprint 0, backlog, Definition of Done, 2-week loops); sensible choice to reuse proven open-source scanners; clear developer-focused user.

| Blind spot found | Why it is a risk | What to do |
|---|---|---|
| No standard schema. The draft invents its own JSON format. | A custom format needs endless upkeep and nothing else can read it. | Use **SARIF v2.1.0**. Trivy, Semgrep, Gitleaks and GitHub Code Scanning already export it. |
| SAST-to-DAST correlation is promised without a method. | A ZAP finding has a URL, a Semgrep finding has a file and line. They share no field. | Build a **route map** (URL to handler function) before promising it. |
| AI fixes may add new flaws. | A model can swap a SQL injection for a weak regex, or break app logic. | **Closed-loop check**: apply the patch in a container, re-scan, run unit tests, then label it Verified Safe. |

### 1.4 Reviewer's recommended 4-sprint roadmap

| Sprint | Deliverable | Acceptance criteria |
|---|---|---|
| **1. SARIF ingestion and CLI** | Containerized CLI runs Trivy, Semgrep, Gitleaks on any git repo and exports unified SARIF. | Scans a benchmark repo (OWASP NodeGoat); clean combined SARIF in under 60 seconds. |
| **2. Dedup and owner router** | PostgreSQL ingestion, AST-based dedup, automatic owner lookup. | 100 raw findings become fewer than 30 unique issues; 100% correct CODEOWNERS attribution. |
| **3. Closed-loop AI patching** | LLM writes a diff; scanner re-runs to verify it. | Diff patches for the top 5 OWASP Top 10 types; fix confirmed by re-scan. |
| **4. Workflow and SLA dashboard** | GitHub Action posts PR comments; small SLA dashboard. | Clean PR comment on new security debt; alerts on overdue SLAs. |

### 1.5 Questions the team must be ready to answer

- How does dedup tell apart two identical secrets found in different branches or different microservice repos? (Include repo and branch in the fingerprint.)
- If Semgrep flags a file inside `node_modules`, who gets it? (Route to the owner of the lockfile / dependency file, not the library.)
- How do we stop proprietary source code leaking to a third-party LLM during fix generation? (Send only the minimal snippet, redact secrets, allow a self-hosted model.)

## 2. From SARIF to the final result, step by step

This is the full journey of one scan. Each step has one job, so a bug in one step is easy to find and fix. The last column shows the sprint where the step is built.

| # | Step | What happens and why | Sprint |
|---|---|---|---|
| 1 | **Scan** | The CLI starts the chosen tools on the repo (Trivy, Semgrep, Gitleaks now; ZAP later, because it needs a running site). Tools run at the same time to save time. | 1 |
| 2 | **Get SARIF** | Trivy, Semgrep and Gitleaks write SARIF with an output flag, so no conversion is needed. ZAP uses its `sarif-json` report template. Tools with no SARIF (for example sqlmap, nmap, custom testers) need a small converter script. The original raw report is always kept too. | 1 |
| 3 | **Validate and merge** | Check each file is real SARIF. A SARIF file holds a list called `runs`, one per tool, so merging means putting all runs into one file. Store repo name, commit ID and scan time with it (needed later for git blame). | 1 |
| 4 | **Parse** | Parsing turns raw text into data the program can use. The parser only **extracts** fields (tool, rule ID, message, level, location, line) into a list of Finding objects. It does not judge or change them. If a tool changes its format, only the parser needs fixing. | 2 |
| 5 | **Normalize** | Make every tool look the same (see the three rules below). | 2 |
| 6 | **Store** | Save the raw report untouched, and the clean findings in PostgreSQL (`jsonb` for the raw part). Index rule ID, location and fingerprint. Track first_seen, last_seen and status (open, fixed, ignored, false positive) so history is never overwritten. | 2 |
| 7 | **Deduplicate** | Layers 1 and 2 of the architecture (next page): same finding seen again, or same finding seen by two similar tools. Duplicates are merged. | 2 |
| 8 | **Correlate** | Layer 3: link a DAST finding (URL) to a SAST finding (file) through the route map. Linked, not merged. | Not yet scheduled |
| 9 | **Assign owner** | Read `.github/CODEOWNERS`; the last matching pattern wins. Fall back to the git blame author if no owner matches. | 2 |
| 10 | **Show result** | CLI summary now. Website and PR comments later, built from the same stored data. | 1 then 4 |

### Step 5 in detail: three normalization rules

- **Location is a URL or a file.** Use one general `location` column plus `location_type` ('file' or 'url'). Line and column stay empty for URLs. Store the URL without query values (`?q=test`) and keep the parameter name (q) in its own column, so two test values do not look like two locations.
- **Severity differs.** ZAP says High/Medium/Low, Semgrep says ERROR/WARNING/INFO, Trivy says CRITICAL to LOW. Pick one scale (critical, high, medium, low, info) and map once at ingestion. Keep the tool's original severity in another column. In SARIF, ZAP's risk may arrive only as the coarse `level`, so check what your version writes.
- **Missing fingerprint.** Some tools give none. Create your own from fields that never change between scans: `hash(tool + rule_id + location + parameter)`. Never include the scan date or changing query values, or the hash changes every time.

```
SARIF input (ZAP):   ruleId=40018   level=error
                      uri=https://app.example.com/api/v1/search
Normalized row:       tool=ZAP  rule_id=40018  severity=high  location_type=url
                      location=https://app.example.com/api/v1/search  param=q  line=null
                      fingerprint=a3f9c1...   raw=<full result as jsonb>
```

### Rules that keep the database clean

- Unique constraint on the fingerprint; NOT NULL on tool, rule_id, severity, location; fixed allowed values for severity and location_type.
- Make ingestion safe to repeat: loading the same file twice must not create duplicates (insert-or-update on the fingerprint).
- Allow human overrides ("not the same bug", "false positive") and respect them on the next scan. Record how every link was made, with a confidence value.

## 3. The three-layer architecture

The question "have I seen this finding before?" has three different answers depending on *who* reported it. Mixing them up is the main source of confusion, so we treat them as three layers. Layers 1 and 2 **merge** findings (they are the same finding). Layer 3 **links** findings (they are two pieces of evidence for one bug) and keeps both rows.

| Layer | Situation | Technique | Result |
|---|---|---|---|
| **1. Same tool, different scans** | Semgrep reports the same bug on Monday and again on Friday. | Hash of (file path, rule ID, target symbol). AST fingerprint plus git blame, so the finding is recognized even when code shifts lines. | Merge |
| **2. Different tools, same category** | Semgrep and CodeQL both flag the same code (both SAST). Or ZAP and another DAST tool flag the same URL. | SAST: same file + same CWE + same function or overlapping lines. DAST: normalized URL + parameter + CWE. No route map needed. | Merge |
| **3. Different categories** | ZAP flags `/api/v1/search`, Semgrep flags `controllers/search.py:42`. | Route map + CWE + correlation rules. A URL and a file share no field, so the route map is the translator. | Link (group id) |

### Layer 1: same tool, different scans

Use a symbol (function name), not a line number, because line numbers move whenever someone edits above them. The AST fingerprint parses the vulnerable code into a syntax tree and hashes its structure, ignoring whitespace, comments and line numbers, so the same code at line 42 or line 52 gives the same hash. `git blame -M -C` shows whether the lines come from the same commit (only moved, not edited) and who wrote them. If someone really edits the vulnerable line, the hash changes and it counts as new, which is usually correct. A big refactor can break the match.

### Layer 2: different tools, same category

Both tools describe the finding in the same language (file to file, URL to URL), so a direct match works. CWE is the shared label for the bug type (CWE-89 is SQL injection), and both tool families tag findings with it.

### Layer 3: SAST versus DAST correlation

```
ZAP     SQL Injection  URL: /api/v1/search?q=test   param: q

Semgrep sql-injection  File: controllers/search.py:42   cursor.execute("..." + q)

Route map:   /api/v1/search [GET]  ->  controllers/search.py::search_handler

1. Normalize the ZAP URL (strip host and query, replace ids with {id})
2. Look up the handler in the route map
3. Find Semgrep findings in that file / function (and its call chain) with the same CWE
4. Save a shared correlation_group_id with the reason and a confidence value
```

Why it is hard: the route is defined in another file (`urls.py` or a decorator), prefixes like `/api/v1` are added by a router elsewhere, the real bug may sit in a helper such as `services/query_builder.py`, and dynamic routes (`/users/{id}`) never match the literal URLs ZAP tests (`/users/17`). Ways to build the map:

| Approach | How | Trade-off |
|---|---|---|
| OpenAPI/Swagger spec | Read paths and operationId | Only as good as the spec |
| Framework introspection | Flask url_map, Django show_urls, Spring /mappings | Accurate but one framework at a time |
| Static route extraction | Rules that find @app.route, path() etc. | Misses dynamic routes |
| Name heuristics | Match URL segments to file or function names | Quick but unreliable |

### Where the logic lives, and what to focus on

The database stores results; the **ingestion code** decides what goes in. A clean schema does not rescue sloppy code. Put **data rules in the database** (constraints, indexes, insert-or-update) and **decision rules in code** (parsing, hashing, AST work, CODEOWNERS matching, anything with if/else). Keep the pipeline as separate functions: `parse, normalize, dedup, correlate, assign_owner, save`, so each can be tested alone. Also: index fingerprint, rule_id, location, correlation_group_id and status; keep raw and normalized data apart; and test with a deliberately vulnerable app (OWASP Juice Shop, DVWA, NodeGoat).

**Gap to raise with the team:** the reviewer's roadmap never schedules ZAP, so Layer 3 is not in any of the four sprints. Decide where it goes (after Sprint 2 is the natural place).

# 4. Sprint 1 in detail: SARIF ingestion and orchestration CLI

**Deliverable:** a containerized CLI that runs Trivy, Semgrep and Gitleaks on any git repository and exports unified SARIF. **Acceptance:** it scans OWASP NodeGoat and produces one clean, combined SARIF file in under 60 seconds. In plain words: one command, any repo in, one SARIF file out, inside Docker, in under a minute.

**Not in Sprint 1:** database, dedup, CODEOWNERS, AI fixes, ZAP, dashboard, GitHub Action. Do not let anyone start these.

## 4.1 CLI blueprint

A terminal tool in the style of OpenCode. It works in any terminal, including the one built into VS Code or IntelliJ, so no IDE extension is needed yet. The user gives the repo, picks one tool or all tools, and sees the time next to each tool name. Results are shown in the CLI now; the website comes later and reads the same data.

```
releaseshield scan <repo-url-or-path>            # menu appears, pick tools
releaseshield scan <repo> --tools trivy           # one tool
releaseshield scan <repo> --tools all --out result.sarif
releaseshield scan <repo> --json                  # machine-readable summary

 Trivy 18.2s done | Semgrep 41.7s done | Gitleaks 6.9s done
 Tool       High  Medium  Low  Total
 Trivy        12      20   10     42
 Semgrep      11      15    5     31
 Gitleaks      3       0    0      3
 Total        26      35   15     76   (total time 42s)   Saved: ./results/result.sarif
```

Show a summary, not every finding. Seventy lines of findings is the same noise the product exists to remove. Total time is the slowest tool when tools run in parallel, not the sum. Severity counts can use the SARIF `level` for now; proper mapping is Sprint 2.

## 4.2 What we build

| Part | Job |
|---|---|
| **Scanner runners (x3)** | One module per scanner: run it, save its SARIF, report ok / issues_found / failed, duration and error. |
| **Orchestrator** | Clone the repo if given a URL, start the chosen runners in parallel, wait, handle failures and timeouts. |
| **SARIF validator** | Reject broken files with a clear message; convert paths to relative from the repo root. |
| **SARIF merger** | Combine the runs into one file; attach repo name, commit ID, time, tool versions. |
| **CLI and display** | Commands, tool menu, progress with times, summary table, saved file path. |
| **Docker image** | CLI plus all three scanners, with vulnerability database and rule sets pre-downloaded. |

| Scanner | Finds | Command (check your installed version) |
|---|---|---|
| **Trivy** | Vulnerable dependencies, container issues | `trivy fs --format sarif --output trivy.sarif <path>` |
| **Semgrep** | Insecure code patterns | `semgrep scan --config <rules> --sarif --output semgrep.sarif <path>` |
| **Gitleaks** | Hardcoded passwords and keys | `gitleaks detect --source <path> --report-format sarif --report-path gitleaks.sarif` |

## 4.3 Problems to expect

- **Exit codes.** Gitleaks (and Semgrep with --error) exit non-zero when they *find something*. That is not a crash. Separate "found issues" from "tool failed".
- **Slow first run.** Trivy downloads a vulnerability database and Semgrep downloads rules. Pre-bake both into the Docker image and pin the rule sets, or the 60 seconds are gone.
- **One scanner fails.** Agree the rule now: the other results are still saved and the output says which tool failed.
- **Path differences.** Absolute versus relative paths between tools; fix during merging. **Language:** use the backend's language (Python if Django, TypeScript if NestJS).

## 4.4 Layout and the one contract everyone follows

```
releaseshield/  cli/  runners/(trivy, semgrep, gitleaks)  sarif/(validator, merger)
                orchestrator/  metadata/  docker/  tests/  docs/

RunnerResult: tool | sarif_path | status (ok, issues_found, failed) | duration | error
```

**Prepare for Sprint 2 now:** keep one run per tool in the merged file, always save repo name and commit ID, and keep every raw scanner output. Nothing is lost, and Sprint 2 needs all three.

# 5. Sprint 1 roadmap, Definition of Done and team roles

## 5.1 Roadmap (2 weeks, 10 working days)

| Days | Focus | What happens |
|---|---|---|
| **1 to 2** | Setup and decisions | Pick language, create repo and folders, set up the board. Agree the RunnerResult contract. Everyone installs the scanners and runs each by hand on NodeGoat and reads the real SARIF. Decide failure handling and output file name. |
| **3 to 5** | Build in parallel | Three runners, validator and merger, CLI skeleton and orchestrator, Dockerfile v1, test plan and benchmark script. |
| **6 to 7** | Join the parts | Connect everything and run end to end on NodeGoat for the first time. Fix what breaks. |
| **8 to 9** | Fast and safe | Measure time and get under 60 seconds. Test failure cases (bad repo, broken SARIF, crashed scanner, empty repo). Finish the README so a new person can run it in 5 minutes. |
| **10** | Finish | Sprint review with a live demo of the acceptance criteria, retrospective, Sprint 2 planning. |

## 5.2 Definition of Done

| | | |
|---|---|---|
| [ ] Works on any git repo | [ ] Valid single SARIF on NodeGoat | [ ] Separate runs for Trivy, Semgrep, Gitleaks |
| [ ] Under 60 seconds (measured) | [ ] Works in Docker on a clean machine | [ ] One failing scanner does not stop the rest |
| [ ] Commit ID and scan time saved | [ ] Unit tests pass | [ ] README written; reviewed and merged to main |

## 5.3 Roles for 12 members

| # | Role | Responsibilities |
|---|---|---|
| 1 | **Product Owner** | Owns the backlog and acceptance criteria, defines "done", keeps scope from growing, runs the demo. |
| 2 | **Scrum Master** | Runs the 15-minute daily stand-up, removes blockers, keeps the board current, runs the retrospective. |
| 3 | **Trivy owner** | Trivy runner and tests; makes its vulnerability database available offline inside Docker. |
| 4 | **Semgrep owner** | Semgrep runner and tests; chooses and pins rule sets; controls its speed. |
| 5 | **Gitleaks owner** | Gitleaks runner and tests; handles its exit-code behavior. |
| 6 | **SARIF validator** | Checks files are valid SARIF, writes clear errors, converts paths to relative. |
| 7 | **SARIF merger** | Combines the runs into one file and attaches the scan metadata (repo, commit ID, time, tool versions). |
| 8 | **CLI developer** | Commands and options, tool menu, progress with times, summary table, `--json` output. |
| 9 | **Orchestrator developer** | Repo cloning, parallel runs, timeouts, failure handling and the RunnerResult contract. |
| 10 | **Docker / DevOps (image)** | Dockerfile with CLI and all scanners; keeps the image small and reproducible. |
| 11 | **Docker / DevOps (speed, CI)** | Pre-downloads databases and rules; automatic build and test; owns the 60-second target. |
| 12 | **QA and documentation** | Runs the NodeGoat benchmark, writes failure tests, measures timing, writes the README. |

## 5.4 Team rules and Day 1 decisions

- Everyone writes unit tests for their own part. Every change goes through a pull request reviewed by someone else.
- Pair dependent people: runners with the orchestrator developer, validator with merger.
- Sprint 1 is small for 12 people. Use spare time to prepare Sprint 2 (draft the PostgreSQL tables, try reading a CODEOWNERS file, research the route map), not to add scope to Sprint 1.
- **Decide on Day 1:** (1) language and framework, (2) output file name and location, (3) failure rule: still produce a result and name the failed tool (recommended), (4) which Semgrep rule sets to pin.